

21. Les variables partagées

Nous souhaitons développer un système de tableaux de sémaphores à l'aide de signaux basé sur un modèle client/serveur. Le client qui souhaite réaliser l'opération P(S_X) envoie le signal SIGRTMIN+1 au serveur avec la valeur X (l'indice du sémaphore dans le tableau). Il attend ensuite de recevoir SIGRTMIN (la valeur associée est 0 en cas de réussite, 1 sinon). S'il souhaite réaliser l'opération V(S_X), il envoie le signal SIGRTMIN+2. Un échec survient si le client envoie un indice incorrect ou si le nombre de clients en attente est trop grand (il s'agit d'une quête optionnelle).

Le serveur prend en arguments les valeurs d'initialisation des sémaphores. Par exemple `./server 1 2` crée deux sémaphores initialisés respectivement à 1 et 2. À chaque sémaphore est associé une valeur et une file d'attente : si un processus est bloqué sur un sémaphore (opération P avec la valeur du sémaphore à 0), son PID est placé dans la file correspondante. Dès qu'un autre processus le débloque (opération V), le serveur envoie un signal pour débloquer le processus en attente. Si une opération peut être réalisée immédiatement (un V ou un P avec une valeur de sémaphore supérieure à 0), le serveur répond immédiatement, sans passer par les files. Pour répondre, le serveur envoie le signal SIGRTMIN avec la valeur 1 (en cas de réussite) ou 0 (en cas d'erreur).

1. Téléchargez l'archive : [archive.zip](#)

L'archive contient 4 fichiers :

- `share.c/share.h` : contiennent des fonctions utilisées pour les clients
- `share_server.c` : les sources de l'application qui permet de créer un tableau partagé
- `share_test.c` : les sources d'une application de test

Vous devez les compléter là où il est indiqué #TODO#. Voici quelques explications :

2. Complétez le fichier `share.c` :

- Dans un premier temps, regardez les structures proposées pour les requêtes et un tableau partagé contenues dans le `share.h`
- `share_init` prend le nom du tableau partagé à créer ; elle commence par bloquer le signal SIGRTMIN+3 qui sera utilisé pour la réception des réponses du serveur ; elle crée le nom du tube nommé et l'ouvre en écriture ; pour récupérer la taille du tableau, elle envoie une requête de taille (en écrivant dans le tube) puis attend la réponse (le signal) à l'aide de `sigwaitinfo`. La valeur reçue est la taille. La structure créée pourra maintenant être utilisée pour les futures requêtes.
- `share_clean` permet simplement de fermer le tube (le descripteur est dans la structure).
- `share_size` récupère la taille du tableau ; comme nous l'avons récupérée dans la fonction `share_init`, il s'agit simplement de retourner la valeur du champ de la structure.
- `share_get` permet de récupérer la valeur d'une des cases du tableau ; elle crée une requête (cette partie est en commentaires pour permettre la compilation), l'écrit dans le tube et attend la réponse (toujours avec `sigwaitinfo`).
- `share_set` est identique, à part la nouvelle valeur de la case que l'on spécifie dans la requête.

3. Complétez le fichier `share_server.c`

- La plupart des éléments sont déjà remplis dans le fichier : la vérification des arguments, la création et l'initialisation du tableau, ainsi que la boucle principale.
- Spécifiez un gestionnaire pour le signal SIGINT ; cela permet d'arrêter la boucle principale (le gestionnaire est déjà présent)
- Créez le tube nommé (le nom est "filename").
- Enregistrez la fonction "end" pour supprimer le tube lorsque l'application s'arrête (`atexit`).
- Ouvrez le tube nommé en lecture.
- Dans la boucle principale :
 - Lisez la requête depuis le tube ; le nombre d'octets lus est important : si c'est 0, c'est qu'il n'y a plus de lecteur et l'application peut s'arrêter.
 - En fonction du type de la requête, envoyez le signal avec la bonne valeur (une partie du code est présente en commentaires).
- Fermez le tube.

4. Testez les applications :

- Lancez le serveur, par exemple : `./share_server t 0`
- Lancez le programme de test, par exemple : `./share_test t`

22. Les sémaphores

Nous souhaitons développer un système de tableaux de sémaphores à l'aide de signaux basé sur un modèle client/serveur. Le client qui souhaite réaliser l'opération P(S_X) envoie le signal SIGRTMIN+1 au serveur avec la valeur X (l'indice du sémaphore dans le tableau). Il attend ensuite de recevoir SIGRTMIN (la valeur associée est 0 en cas de réussite, 1 sinon). S'il souhaite réaliser l'opération V(S_X), il envoie le signal SIGRTMIN+2. Un échec survient si le client envoie un indice incorrect ou si le nombre de clients en attente est trop grand (il s'agit d'une quête optionnelle).

Le serveur prend en arguments les valeurs d'initialisation des sémaphores. Par exemple `./server 1 2` crée deux sémaphores initialisés respectivement à 1 et 2. À chaque sémaphore est associé une valeur et une file d'attente : si un processus est bloqué sur un sémaphore (opération P avec la valeur du sémaphore à 0), son PID est placé dans la file correspondante. Dès qu'un autre processus le débloque (opération V), le serveur envoie un signal pour débloquer le processus en attente. Si une opération peut être réalisée immédiatement (un V ou un P avec une valeur de sémaphore supérieure à 0), le serveur répond immédiatement, sans passer par les files. Pour répondre, le serveur envoie le signal SIGRTMIN avec la valeur 1 (en cas de réussite) ou 0 (en cas d'erreur).

1. Téléchargez l'archive : [archive.zip](#)

Dans cette archive, il y a plusieurs fichiers :

- `queue.c` et `queue.h` : une implémentation de file de PID ; ces fichiers sont complets
- `dijkstra.c` et `dijkstra.h` : les fonctions liées aux sémaphores pour les clients
- `server.c` : les sources de l'application qui crée un tableau de sémaphores
- `client.c` : un client qui teste les sémaphores (c'est une simple exclusion mutuelle)

L'objectif est de compléter les fichiers `dijkstra.c/dijkstra.h` et `server.c` là où il y a des `#TODO#`, en suivant les directives :

2. Fichiers `dijkstra.h` et `dijkstra.c` :

- Commencez par regardez le fichier `dijkstra.h` avec la structure et les signatures de méthodes ; la structure `sem_t` correspond simplement au PID du serveur ; il serait possible d'y ajouter d'autres choses (comme la taille du tableau de sémaphores, par exemple).
- `sem_init` vérifie si le serveur existe (le PID est passé en paramètre) ; si c'est le cas, on initialise la structure puis on bloque les signaux SIGRTMIN qui seront utilisés pour recevoir les réponses du serveur
- `sem_clean` : rien à faire...
- `sem_P` correspond à l'opération P sur un sémaphore dont le numéro est passé en paramètre ; on envoie un signal SIGRTMIN+1 au serveur avec le numéro puis on attend l'accusé de réception (à l'aide de `sigwaitinfo`) qui bloque tant que le sémaphore ne peut pas être décrémenté
- `sem_V` correspond à l'opération V (le signal est SIGRTMIN+2)

3. Fichier `server.c` :

- Toutes les fonctionnalités de base sont déjà complétées : la vérification des arguments, la récupération des valeurs, l'allocation du tableau correspond au tableau de sémaphores
- On doit dans un premier temps bloquer les signaux SIGRTMIN+1 et SIGRTMIN+2 qui seront envoyés par les clients
- Dans la boucle principale :
 - On attend un signal SIGRTMIN+1 ou SIGRTMIN+2
 - Si c'est le signal SIGRTMIN+1, c'est une opération P ; si la valeur du sémaphore est supérieure à 0, alors on envoie l'accusé à l'émetteur directement ; dans le cas contraire, on doit empiler le PID (sauf si la pile est pleine)
 - Si c'est le signal SIGRTMIN+2, c'est une opération V ; on incrémente la valeur du sémaphore ; on envoie l'accusé de réception directement ; ensuite on regarde si la pile n'est pas vide : si c'est le cas, on doit débloquer le premier de la file (on récupère son PID de la file, on envoie un signal).
 - Si le numéro de sémaphore n'est pas valide, on envoie un signal d'erreur (c'est une quête secondaire)

4. Testez les programmes :

- Lancez le serveur : `./server 1` (cela crée un tableau de 1 sémaphore initialisé à 1)
- Lancez plusieurs clients : `./client PID` (où PID est le PID du serveur qu'il a affiché)
- Vérifiez qu'un seul client à la fois entre dans sa section critique

23. Le problème des philosophes

Nous souhaitons utiliser les variables partagées et les sémaphores développées précédemment pour résoudre le problème des philosophes vu en cours.

1. Quelle(s) variable(s) partagée(s) avez-vous besoin ? Il est conseillé d'utiliser un seul tableau.
2. Quelle(s) sémaphore(s) avez-vous besoin ? Il est conseillé d'utiliser un seul tableau de sémaphores.
3. Écrivez philosophe.c qui simule un philosophe ; il prend en arguments le PID du serveur de sémaphores et le nom de la variable partagée, ainsi que son numéro (de 1 à 5).
4. Vérifiez le bon fonctionnement de votre programme.